

EM Inversion Workshop

GUI Manual

This manual describes how to set up a Python environment, install dependencies, and run the six workshop GUIs. For each GUI it explains how to launch it, how to set parameters, and—for non-expert users—what each parameter means physically and what impact it has on the simulation or workflow.

The six GUIs are:

- **01_fw_setup** — Prepare FD modelling inputs from a SEG-Y file.
- **02_visualization** — Run FD modelling and visualize channel-gain amplitude/phase from shot records.
- **03_inversion** — Generate inversion inputs and run the 2D inversion locally.
- **04_results** — View 2D inversion results and compare real data with synthetics.
- **05_1d_empymod_inversion** — Independent 1D layered inversion, per transmitter, against a validated analytic magnetic (Hx) line-source solver (despite the filename, no `empymod` is used).
- **06_1d_empymod_results** — View the 1D inversion’s pseudo-2D section, per-Tx diagnostics, and modelled-vs-observed data comparison.

GUIs 01–04 form the 2D explicit-FDTD forward/inversion workflow; 05–06 are an independent, complementary 1D analysis that runs directly against the 2D forward run’s own data (no separate FDTD run needed) and does not depend on 03/04’s inversion having been run.

All GUIs are Jupyter notebooks served as code-hidden applications via Voila (ipywidgets and Plotly). Run Voila from the **workshop root** so that **scripts** and **third_party** are on the Python path.

1 Environment setup

1.1 System prerequisites

Before setting up Python, ensure you have:

- Python 3.10 or newer
- `mpirun` (OpenMPI or compatible MPI runtime)
- A validated `rockem-suite` checkout, built with `make mpi`, with the **explicit** TE2D engine binaries `mpiEmmodTE2d` and `mpiEminvTE2d` under its `bin/` directory. Their location is `$ROCKEM_SUITE_ROOT/bin`, where `ROCKEM_SUITE_ROOT` is an environment variable defaulting to `~/software/new_rockem/rockem-suite` (see `scripts/modules/rockem_bridge.py`).

The ADI TE2D engine (`mpiEmmodADITE2d/mpiEminvADITE2d`) is **not** used by this workshop — it fails rockem-suite’s own layered-model Green’s-function validation.

1.2 Creating a virtual environment (venv)

From the workshop root:

```
python3 -m venv .venv
source .venv/bin/activate    (Linux/macOS)
.venv\Scripts\activate      (Windows)
```

Then upgrade pip: `pip install -upgrade pip`.

1.3 Creating a conda environment

```
conda create -n em_workshop python=3.10
conda activate em_workshop
```

1.4 Installing dependencies

With the virtual or conda environment activated, install:

Package	Purpose
voila	Serves notebooks as standalone web apps (code hidden).
ipywidgets	Interactive widgets in the GUI.
plotly	Interactive plots.
numpy	Numerical arrays (used by inversion and results GUIs).
ipykernel	Jupyter kernel for Python (required for Voila to run notebooks).
matplotlib	Plotting (used by workshop modules).
scipy	Scientific computing (used by workshop modules).
segio	Read/write SEG-Y files (required by 01_fw_setup).
joblib	Parallelizes 05’s Tx × seed ensemble inversion across CPU cores.

Command:

```
pip install voila ipywidgets plotly numpy ipykernel matplotlib scipy
segio joblib
```

Conda users: After installing, register the kernel so Voila can find it:

```
python -m ipykernel install -user -name=em_workshop -display-name="Python
(em_workshop)"
```

For reproducibility you can add a `requirements.txt` in the project root with these packages and then run `pip install -r requirements.txt`.

2 How to run the GUIs

From the **workshop root** directory, use the provided shell scripts. Each script changes into the script directory and starts Voila with the corresponding notebook (sources stripped so users see only the interface).

- `./start_01_fw_setup.sh` → 01_fw_setup
- `./start_02_visualization.sh` → 02_visualization
- `./start_03_inversion.sh` → 03_inversion

- `./start_04_results.sh` → 04_results
- `./start_05_1d_empymod_inversion.sh` → 05_1d_empymod_inversion
- `./start_06_1d_empymod_results.sh` → 06_1d_empymod_results

Run 05/06 after 01/02 have produced FD data for at least one transmitter — they read directly from that data and do not require 03/04’s 2D inversion to have been run.

Alternatively, from the workshop root:

```
voila 01_fw_setup.ipynb -strip_sources=True
```

(and similarly for the other five notebooks.)

Note: 01_fw_setup and 04_results both use `.voila_server.pid`; avoid running both at the same time if you rely on that PID file.

3 GUI 1: 01_fw_setup

3.1 Purpose

Generate finite-difference (FD) modelling inputs from a SEG-Y resistivity model: conductivity/permittivity files (`sg.rss`, `ep.rss`), source wavelet (`wav2d.rss`), survey geometry (`Survey.rss`), and `mod.cfg` settings. Intermediate files are written to a temporary session folder; final outputs go to `FDmodel`.

3.2 Parameters (physical meaning and impact)

- **SEG-Y file** — Path to the input SEG-Y file containing the resistivity model. This defines the subsurface you are simulating: the grid and resistivity values control where EM waves propagate and how they attenuate. Use a model that matches the geology and scale of your survey.
- **ep constant** — Electrical permittivity (relative dielectric constant) assigned uniformly in the model. It affects wave speed and wavelength. It is usually kept constant when the focus is on conductivity/resistivity. Changing it shifts phase and can affect how well synthetics match real data if the data are sensitive to permittivity.
- **tx0 (m), tz0 (m), dtx (m), ntx** — Transmitter (source) geometry: first source position in x and z , spacing between sources, and number of sources. This is where the EM signal is “injected.” All positions must lie inside the model bounds. More sources give more data but increase run time.
- **rx0 (m), rz0 (m), drx (m), nrx** — Receiver geometry: first receiver x and z , receiver spacing, and number of receivers. This is where the field is “measured.” Spacing controls spatial sampling and resolution; typically receivers are offset from the source.
- **flist (Hz)** — Comma-separated list of source frequencies (e.g. 2000,4000,6000). Lower frequencies penetrate deeper but have lower resolution; higher frequencies resolve shallow structure better. Choose frequencies that match the content you want to invert or compare with data.
- **dt (s)** — Time step for the wavelet and for the FD solver. It must be small enough for numerical stability (CFL) and to sample the highest frequency. Too large a value causes instability or numerical dispersion; too small increases run time and memory.

- **n_periods** — Number of periods of the lowest frequency in the wavelet. This controls how long the source radiates and affects the amplitude and phase of the synthetic data.
- **alpha** — Ramp parameter (0–1) for the ramp square-wave wavelet. It shapes the rise and fall of the source and affects which frequencies dominate in the synthetic.
- **f_min (Hz), f_max (Hz)** — Frequency range used to design the FD grid for the **explicit** engine (`mpiEmmodTE2d`). Everything below (grid spacing, time step, **eps_r**, PML, depth margin) follows deterministically from these plus **rho_min/rho_max** and the receiver offsets — there is no separate $dx/dt/CFL$ widget to hand-tune, since these are numerical consequences of the physics, not independent choices.
- **rho_min (Ohm-m), rho_max (Ohm-m)** — Expected resistivity range in the model. Grid spacing is sized from the more restrictive of skin depth (at **f_max/rho_min**) and the minimum receiver offset; the **eps_r** used to relax the explicit engine’s CFL time-step limit is sized from **rho_max/f_max**. Set them to bracket your actual model to avoid undersampling or unnecessary refinement.
- **dimension** — 2D or 3D. The FD solver uses a 2D or 3D grid accordingly. 2D is much faster and is appropriate for line surveys. **Note:** the 3D path still uses the legacy ADI engine and is not covered by this workshop’s explicit-engine validation — treat 3D results as unvalidated.
- **points/skin** — Number of grid points per skin depth at the worst case (highest conductivity, highest frequency). More points improve accuracy but increase grid size and run time; too few cause numerical dispersion and wrong amplitudes/phases.
- **cells/min offset** — Number of grid cells across the smallest receiver offset. For near-offset surveys this (not skin depth) often ends up the more restrictive constraint on grid spacing — see **points/skin** above; the FD design uses whichever of the two implies the finer grid.
- **tan(delta) floor** — Minimum loss-tangent ($\sigma/(\omega\epsilon)$) the design will tolerate at the least-conductive/highest-frequency corner of the survey when choosing **eps_r**. Raising **eps_r** relaxes the explicit engine’s CFL time-step limit (bigger time steps, faster runs) as long as the medium stays comfortably conduction-dominated; this floor caps how far **eps_r** is pushed so that trade-off stays physically negligible.
- **eps_r cap** — Hard ceiling on the **eps_r** the design will choose, regardless of what the **tan(delta)** floor would otherwise allow. The FD design readout reports whether this cap is binding.
- **sigma_max clip (S/m, 0=off)** — Optional cap on the conductivity used when sizing the grid/**eps_r**, for models with a few extremely conductive outlier cells that would otherwise force an unnecessarily fine grid. Leave at 0 to disable.
- **Depth margin (auto)** — Not a separate widget: the FD design readout also reports the minimum clearance the model needs, in depth, from the source/receiver depths to the top/bottom of the FD domain before the absorbing boundary. If the loaded model falls short, **Generate FD inputs** automatically pads it (extending the shallowest/deepest layer outward) and reports how much was added — a model that’s too shallow there can otherwise give a stable but measurably wrong result with no other warning.
- **Buttons** — “Load model” loads the SEG-Y and updates resistivity bounds; “Plot model” refreshes the resistivity plot; “Build intermediate sg/ep” writes conductivity and permittivity files; “Build intermediate wav2d” and “Plot wavelet + spectrum” create and visualize the wavelet; “Generate survey.cfg + Survey.rss” writes survey geometry; “Compute FD design” computes recommended grid spacing, time step, **eps_r**, PML, and depth margin;

“Generate FD inputs (Finalize setup)” interpolates to the target grid, pads depth if needed, and writes everything to `FDmodel`. Use “Quit GUI server” to stop Voila.

4 GUI 2: 02_visualization

4.1 Purpose

Run FD modelling (via `FDmodel/runmod.sh` and the explicit-engine MPI binary), load `Hxshot.rss` and `Hzshot.rss` outputs, compute steady-state channel-gain amplitude and phase, and visualize or save the results.

4.2 External requirement

The explicit-engine MPI FD binary (`$ROCKEM_SUITE_ROOT/bin/mpiEmmodTE2d`) must be installed and `FDmodel/runmod.sh` must resolve `ROCKEM_SUITE_ROOT` correctly (default `~/software/new_rockem/rockem-suite`). The GUI and run script reference this binary.

4.3 Parameters (physical meaning and impact)

- **nproc** — Number of MPI processes used to run the FD modelling. More processes typically shorten wall-clock time if you have enough cores; it does not change the physics of the result.
- **f_min (Hz)** — The lowest frequency in the recorded wavelet, used as the reference for the steady-state phasor extraction window (see **n_periods** below) — every frequency’s window is scaled from this one so a single shared Hann window covers the whole multi-tone recording without tones leaking into each other’s estimate.
- **n_periods (of f_min)** — Number of periods of **f_min** used as the extraction window (applied to every frequency, scaled by its ratio to **f_min**). The window must be well within the steady-state part of the recording, past the source’s startup ramp; too short a window leaves transient energy in the estimate.
- **component** — Hx or Hz: which magnetic field component is plotted (in-line or vertical). Different components have different sensitivity to structure and are used for different interpretation or inversion choices.
- **plot (metric)** — Which quantity is plotted: amplitude or phase vs receiver, vs transmitter, or vs frequency. Amplitude reflects attenuation and geometric spreading; phase reflects travel path and velocity. Choose the view that matches your interpretation or data comparison.
- **frequency, tx, local rx** — Selectors for which frequency, source, or receiver index to display. Use them to inspect specific source–receiver pairs or frequencies relevant to your interpretation.
- **Buttons** — “Run modelling (background)” starts the FD run; “Stop run” stops it; “Refresh run status” updates job progress; “Load FD outputs” loads `Hxshot.rss/Hzshot.rss`; “Compute channel-gain amplitude/phase” runs the steady-state phasor-ratio extraction (trace phasor over injected-wavelet phasor, so any convention-dependent scale cancels — no time-derivative or phase-correction constants); “Save processed results” writes the NPZ; “Quit GUI server” stops Voila.

5 GUI 3: 03_inversion

5.1 Purpose

Generate inversion inputs from `FDmodel` and the inversion template, run the inversion locally (staged into the next `InversionRunN` directory), and monitor logs and model progress.

5.2 External requirement

The explicit-engine MPI inversion binary (`$ROCKEM_SUITE_ROOT/bin/mpiEminvTE2d`) must be installed and `scripts/templates/runinv.sh` must resolve `ROCKEM_SUITE_ROOT` correctly (default `~/software/new_rockem/rockem-suite`). The GUI and run script reference this binary.

5.3 Parameters (physical meaning and impact)

- **Initial model** — Whether the starting model is defined by uniform conductivity (`sigma`) or uniform resistivity (`rho`). The physics is the same ($\sigma = 1/\rho$); the choice is for convenience. A good starting value is often a background estimate from your data or prior knowledge.
- **sigma (S/m), rho (Ohm.m)** — Value of the uniform initial model. If it is too far from the true structure, the inversion may need more iterations or get stuck; if it is reasonably close, convergence is faster and more stable.
- **Max iter** — Maximum number of inversion iterations. More iterations allow a better fit but increase run time; too few may leave the model under-fit.
- **apertx (m), dtx, dtz** — Grid and extent parameters passed to the inversion. `apertx` defaults to the forward run's own value (from `01_fw_setup`'s FD design) when available, so the inversion's local model isn't accidentally narrower than what the forward run used. All three should stay consistent with the FD model used to generate synthetics so that the forward and inverse use the same discretization.
- **Tikhonov (sg)** — Regularization weight (`tik_sg`regalpha) applied by the inversion engine to discourage overly rough conductivity updates between iterations. Zero disables it; larger values favor smoother models at the cost of fitting the data less exactly.
- **Clean selected InversionRunN before run** — If checked, the chosen run directory is cleared before staging new inputs. Use this to avoid mixing old files when re-running; uncheck to preserve or append to an existing run.
- **Load run** — Select which `InversionRunN` to monitor or use as context. Lets you switch between runs to compare progress or results.
- **nproc** — Number of MPI processes for the inversion run. More processes reduce wall-clock time when you have enough cores.
- **Buttons** — “Generate inversion inputs” writes inputs to `InversionInput/`; “Run inversion locally” stages and starts the inversion; “Refresh from progress.log” updates the plot and logs; “Stop run” stops the process; “Quit GUI server” stops Voila.

6 GUI 4: 04_results

6.1 Purpose

View inversion results: select a run and model (e.g. `sg0`, `sg_1s` at an iteration), view 2D resistivity with true-model comparison, resistivity vs depth or vs x slices, compare real data with synthetics

(either loaded from the run or generated by running FD for the selected model), and export selected resistivity models to SEG-Y.

6.2 Parameters (physical meaning and impact)

- **Run, Model** — Which inversion run and which model file (e.g. initial `sg0`, or `linesearch sg_ls` at an iteration). Use these to inspect how the model evolved and to compare with the true model.
- **Export selected model to SEG-Y** — Writes the currently selected model to `InversionRunN/SEG_Y_output/` using the SEG-Y from setup as a template (same header layout, trace/sample organization, origin, and sampling). If the model grid differs from the template grid, the GUI interpolates to the template grid before writing so exported SEG-Y matches the setup SEG-Y geometry exactly.
- **Export naming and overwrite behavior** — Exported files are deterministic and overwrite on repeated export of the same model: `sg_up.rss-11` → `sg_up_iter011.segy`, `sg0.rss` → `sg0_iter000.segy`. For `sg_ls.rss`, the GUI uses the latest iteration seen in `progress.log` when available (`sg_ls_iterNNN.segy`), otherwise `sg_ls_iterunknown.segy`.
- **Export status (near button)** — A dedicated status banner is shown directly below the run/model/export controls. It reports in-progress, success, or failure, including output filename and folder, so users do not need to scroll to verify export results.
- **x (m), z (m)** — Positions for vertical and horizontal resistivity slices. Use them to see resistivity vs depth at a given x , or vs horizontal position at a given depth, for interpretation.
- **Freqs (Hz)** — Frequencies used when loading real data or generating/running synthetics. They must match the frequencies in your survey and inversion for a meaningful comparison.
- **Load real data / Load synthetics from run / Generate synthetics from model** — Real data are your observations; “Load synthetics from run” loads synthetics saved during inversion; “Generate synthetics from model” runs the FD solver for the selected model so you can test that model without re-running the full inversion. Use these to compare observed vs predicted and assess fit.
- **nproc** — Number of MPI processes when “Generate synthetics from model” is used; it only affects the run time of that FD job.
- **component, plot, frequency, tx, local rx, trace idx** — Same interpretation as in `02_visualization`: choose the component and plot type that match your comparison or interpretation needs.
- **Quit GUI server** — Stops the Voila server.

7 GUI 5: 05_1d_empymod_inversion

7.1 Purpose

An independent 1D layered inversion, run separately for each transmitter, against rockem-suite’s validated analytic magnetic (Hx) line-source Green’s function — despite the filename, no `empymod` is used; it was replaced during this workshop’s redesign because reconciling 2D FDTD data against `empymod`’s 3D point-dipole solution required a stack of ad-hoc correction constants that the analytic line-source solver makes unnecessary. This GUI reads channel-gain data directly

from the 2D forward run’s own `FDmodel/Data` outputs (via the same steady-state phasor extraction as `02_visualization`) — it does not need `03/04`’s 2D inversion to have been run first. Each transmitter is calibrated against the analytic solver evaluated on the exact blocky 1D model beneath it (read from that run’s own `sg.rss`), then inverted independently; results are combined into a pseudo-2D section.

7.2 Parameters (physical meaning and impact)

- **Freqs (Hz)** — Frequencies to extract channel-gain features at. Must match frequencies actually present in the forward run’s wavelet.
- **n_periods (of f_min)** — Steady-state extraction window length, same convention as `02_visualization`.
- **n_layers** — Number of layers in the 1D model parameterization (the last is always a half-space). More layers can fit finer depth structure but add search dimensions and increase the risk of trading off resistivity against thickness in a way the data can’t actually resolve.
- **z_start, z_end** — Depth window (relative to the transmitter depth) that the layered model spans. Should bracket the depth range you expect meaningful structure in; layers are free to redistribute within this span.
- **rho_bg** — Background resistivity used to build default depth-window bounds; not itself a search parameter.
- **rho_min, rho_max, thk_min, thk_max** — Search bounds (uniform in $\log_{10}$) for each layer’s resistivity and thickness. Set these to bracket physically plausible values for your survey — the optimizer searches only within this box.
- **optimizer** — *Differential Evolution* and *Dual Annealing* (both from SciPy) are global stochastic optimizers; *BlockInv* is a deterministic, pyGIMLi-style Gauss–Newton block inversion. The stochastic optimizers explore the bounded search space more broadly (useful with little prior information) at higher computational cost; *BlockInv* converges faster from a reasonable starting point but can get stuck in a local minimum.
- **calibration** — *Fixed calibration constant* (default, recommended) measures one complex constant per frequency once, from the known true model beneath each transmitter, and reuses it for every candidate model tried during that transmitter’s search — appropriate because this constant reflects the FD engine’s own source-injection/grid-discretization convention, not the specific resistivity structure (confirmed by direct testing across several different FD designs and resistivity regimes). *Per-frequency source factor* instead re-solves the same closed-form fit fresh for every candidate model — a fallback for when the fixed constant is found to be unstable (e.g. too few receivers per transmitter to pin it down reliably), at the cost of letting the optimizer partially reabsorb genuine model-scale misfit into the calibration itself.
- **maxiter, popsize, maxfun** — Differential Evolution’s iteration count, population size, and function-evaluation cap. Larger values search more thoroughly at proportionally higher cost. (`maxfun` instead bounds Dual Annealing’s evaluation count.)
- **seed** — Base random seed. With `N runs/Tx > 1`, each repeat uses `seed + run_index`, giving an ensemble of independent optimizer restarts per transmitter used to estimate uncertainty (mean/std across the ensemble), not just a single point estimate.
- **N runs/Tx** — Number of independent restarts per transmitter (see `seed` above). More runs give a better-sampled uncertainty estimate at proportionally higher cost.

- **n_jobs (-1=all cores)** — Every (transmitter, run) combination is fully independent, so the whole ensemble is parallelized across CPU cores via `joblib`. -1 uses all available cores; set a smaller positive number to leave cores free for other work. This does not change results, only wall-clock time.
- **blk_iter** — Maximum Gauss–Newton iterations for the BlockInv optimizer (ignored by the other two).
- **w_hxh, w_hxhz** — Relative weights of the Hx and Hz channel-gain misfit terms. Increase **w_hxhz** relative to **w_hxh** (default 2:1) to trust Hz more; note Hz can be genuinely small/uninformative near a source-depth or zero-offset null (see 06’s per-Tx diagnostics), where it is automatically down-weighted through its own noise estimate regardless of this setting.
- **Tikhonov lambda** — Smoothness penalty on *adjacent-layer* log10-resistivity differences (not a penalty toward any background/reference value) — discourages the optimizer from inventing sharp, unsupported layer-to-layer contrasts to chase noise below the solver’s own validated accuracy floor. Zero disables it.
- **dx_out, dz_out, zmax (0=auto)** — Output grid spacing (horizontal/vertical) and maximum depth for the pseudo-2D section built from all transmitters’ 1D results; **zmax=0** auto-selects a depth based on the inverted models.
- **Buttons** — “Reload features” (re-)extracts channel-gain data from the FD run; “Check kx convergence (prior spot-check)” spot-checks the analytic solver’s internal quadrature resolution across your prior bounds before committing to a full run; “Run all Tx inversions (calibrates per-Tx)” runs the full ensemble described above; “Build pseudo-2D section” combines per-Tx results; “Export results” writes results out; “Refresh QC plots” (QC Tx / QC freq selectors) shows per-transmitter, per-frequency diagnostic plots; “Quit GUI server” stops Voila.

8 GUI 6: 06_1d_empymod_results

8.1 Purpose

Visualize and export the 1D analytic-solver ensemble inversion results from 05: the pseudo-2D section (mean and uncertainty), per-transmitter model diagnostics with error bars, and modelled-vs-observed data comparison — either against synthetics generated from the inverted model or against real data.

8.2 Parameters (physical meaning and impact)

- **Run** — Which `OneDInversionRunN` (under `Results/Analytic1D/`) to load.
- **x slice (m), z slice (m)** — Positions for vertical and horizontal resistivity slices through the pseudo-2D section.
- **bg rho (Ohm.m)** — Background resistivity used only when rebuilding the section display from saved per-layer results.
- **Tx** — Which transmitter’s per-Tx diagnostics (resistivity-vs-depth with uncertainty, misfit) to display.
- **Uncertainty** — *1 std* or *90% CI*: which uncertainty band to draw around the ensemble mean in the per-Tx and section plots.

- **Freqs (Hz)** — Frequencies used when loading real data or generating synthetics from the inverted model, for the data-comparison panel.
- **component, plot (metric), frequency, local rx, trace idx** — Same interpretation as in 02_visualization/04_results: choose the component and view that match your comparison.
- **Buttons** — “Load run”/“Refresh run list” select and (re)load a `OneDInversionRunN`; “Re-build section from saved layers” recomputes the pseudo-2D section display; “Refresh Tx plots” updates the per-Tx diagnostic panel; “Load real data” loads observed channel-gain data for comparison; “Generate synthetics from inverted model” runs the analytic solver on the recovered model for that comparison; “Export mean section to SEG-Y” writes the ensemble-mean resistivity section using the setup SEG-Y as a template; “Quit GUI server” stops Voila.

9 Troubleshooting

- **Missing segyio** — If `01_fw_setup` fails to import workshop modules, ensure `segyio` is installed and that Voila is started from the workshop root so `scripts` and `third_party` are on the path.
- **No Jupyter kernel for language ‘python’ found** — Install `ipykernel` and register the kernel (see Install dependencies above). Conda users must run `python -m ipykernel install -user -name=em_workshop` after installing packages.
- **SEG-Y export cannot find template** — `04_results export` expects a setup SEG-Y template from `FDmodel/setup_metadata.json` (`segy_template_path`), with fallback to `input.segy` in the project root. Run `01_fw_setup` and finalize setup first, or place a valid `input.segy` in the root.
- **Missing explicit-engine MPI FD binary** — `02_visualization` and `04_results` (when generating synthetics) need `$ROCKEM_SUITE_ROOT/bin/mpiEmmodTE2d`. Check that `ROCKEM_SUITE_ROOT` (default `~/software/new-rockem/rockem-suite`) points at a checkout built with `make mpi`, and that `FDmodel/runmod.sh` resolves it correctly.
- **Missing explicit-engine MPI inversion binary** — `03_inversion` needs `mpiEminvTE2d` under `$ROCKEM_SUITE_ROOT/bin`. Check `scripts/templates/runinv.sh` the same way.
- **Wrong/stale rockem-suite checkout** — if results don’t match this manual’s validated accuracy claims, confirm `ROCKEM_SUITE_ROOT` isn’t pointing at an old checkout that pre-dates this workshop’s redesign (e.g. one missing `magnetic_line_source_fields_layered` in `doc/examples/validate_layered_1d_model/shared/greens_layered_2d.py` — `scripts/modules/rockem_bridge.py` raises a clear error in that case).
- **Missing joblib** — `05’s ensemble inversion` needs `joblib` to parallelize the Tx × seed batch (see Install dependencies above); without it the import fails at the top of the notebook.
- **PID file** — `01_fw_setup` and `04_results` both use `.voila_server.pid`; do not run both at once if you rely on that file.

10 File locations

- `FDmodel/` — FD inputs and outputs (e.g. `mod.cfg`, `sg.rss`, `Survey.rss`, `Data/`).
- `InversionInput/` — 2D inversion input files (e.g. `inv.cfg`, `sg0.rss`).
- `InversionRun0/`, `InversionRun1/`, ... — Staged 2D inversion run directories (03/04).

- `InversionRunN/SEG_Y_output/` — SEG-Y exports generated from 04_results for selected models.
- `Results/Analytic1D/OneDInversionRun0/`, ... — 1D inversion run directories (05/06), independent of `InversionRunN/`.
- `scripts/modules/` — Python helpers imported by all six notebooks (SEG-Y I/O, FD design, the rockem-suite bridge, the 1D analytic forward/calibration, 2D inversion helpers); see `scripts/README.md` for the full module map.
- `scripts/templates/` — Templates for `inv.cfg`, `mod.cfg`, `mod3d.cfg`, `survey.cfg`, `runmod.sh`, `runinv.sh`, `clean.sh`.
- `third_party/` — Vendored Rockseis Python code and the BlockInv (pyGIMLi-style) 1D inversion helpers used by the workshop.